

Міністерство освіти та науки України

Національний університет “Львівська політехніка”

Інститут інформаційно-комунікаційних технологій та електронної
інженерії



ЗВІТ

З Лабораторної роботи N8
з дисципліни “Об’єктно-орієнтовне програмування частина 2”

Виконав:

Ст. гр. ІХ-22

Ненадовський Р.М.

Прийняв:

Плесканка М.В.

Лівів 2026

Тема: Використання SQL для роботи з даними.

Мета: Навчитися виконувати SQL-запити, використовуючи SQLite та Pandas для управління реляційними даними.

Завдання:

Реалізувати MQTT-клієнт.

Основні задачі:

- підключення до брокера;
- публікація повідомлень у тему;
- обробка подій (callback);
- відключення від брокера.

Код:

```
import asyncio
import requests
import websockets
import paho.mqtt.client as mqtt

# =====
# MQTT CLIENT
# =====
class MQTTClient:
    def __init__(self, broker, port):
        self.broker = broker
        self.port = port
        self.client = mqtt.Client()

        self.client.on_connect = self.on_connect
        self.client.on_disconnect = self.on_disconnect

    def on_connect(self, client, userdata, flags, rc):
        if rc == 0:
            print("✅ Підключено до MQTT брокера")
        else:
            print(f"❌ Помилка підключення: {rc}")

    def on_disconnect(self, client, userdata, rc):
        print("⚡ Відключено від брокера")

    def connect(self):
        self.client.connect(self.broker, self.port, 60)
        self.client.loop_start()

    def publish(self, topic, message):
        result = self.client.publish(topic, message)
        result.wait_for_publish()
        print(f"📡 MQTT → {message}")

    def disconnect(self):
        self.client.disconnect()
        self.client.loop_stop()

# =====
```

```
# REST API

def get_data_from_api():
    try:
        url = "https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT"
        response = requests.get(url, timeout=5)
        data = response.json()

        price = data["price"]
        return f"Bitcoin price: {price}"

    except Exception as e:
        print("❌ Помилка API:", e)
        return "API error"
```

```
# WEBSOCKET SERVER
clients = set()

async def websocket_handler(websocket):
    print("🟢 Клієнт підключився")
    clients.add(websocket)

    try:
        async for message in websocket:
            print(f"📩 WS отримано: {message}")
    finally:
        clients.remove(websocket)
        print("🔴 Клієнт відключився")

async def send_to_clients(message):
    if clients:
        await asyncio.gather(*[client.send(message) for client in clients])
```

```

# MAIN

BROKER = "broker.hivemq.com"
PORT = 1883
TOPIC = "lab8/test"

mqtt_client = MQTTClient(BROKER, PORT)

async def main():
    print("🚀 Запуск сервера...")

    # WebSocket
    server = await websockets.serve(websocket_handler, "localhost", 8765)

    # MQTT
    mqtt_client.connect()

    try:
        while True:
            data = get_data_from_api()
            print(f"🌐 API → {data}")

            await send_to_clients(data)
            mqtt_client.publish(TOPIC, data)

            await asyncio.sleep(5)

    except KeyboardInterrupt:
        print("🛑 Зупинка...")

    finally:
        mqtt_client.disconnect()
        server.close()
        await server.wait_closed()

# =====

# START

if __name__ == "__main__":
    asyncio.run(main())

```

Результат:

```

✅ Підключено до MQTT брокера
🌐 API → Bitcoin price: 78939.67000000
📡 MQTT → Bitcoin price: 78939.67000000
🌐 API → Bitcoin price: 78939.68000000
📡 MQTT → Bitcoin price: 78939.68000000
🌐 API → Bitcoin price: 78939.68000000
📡 MQTT → Bitcoin price: 78939.68000000
🌐 API → Bitcoin price: 78923.20000000
📡 MQTT → Bitcoin price: 78923.20000000
🌐 API → Bitcoin price: 78923.20000000
📡 MQTT → Bitcoin price: 78923.20000000
🌐 API → Bitcoin price: 78923.20000000
📡 MQTT → Bitcoin price: 78923.20000000
🌐 API → Bitcoin price: 78923.21000000
📡 MQTT → Bitcoin price: 78923.21000000
🌐 API → Bitcoin price: 78923.21000000
📡 MQTT → Bitcoin price: 78923.21000000

```

```
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78901.25000000
MQTT → Bitcoin price: 78901.25000000
API → Bitcoin price: 78901.25000000
MQTT → Bitcoin price: 78901.25000000
API → Bitcoin price: 78901.25000000
MQTT → Bitcoin price: 78901.25000000
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78901.25000000
MQTT → Bitcoin price: 78901.25000000
API → Bitcoin price: 78901.26000000
MQTT → Bitcoin price: 78901.26000000
API → Bitcoin price: 78915.94000000
MQTT → Bitcoin price: 78915.94000000
```

Висновок: Поєднання SQL та Pandas оптимізує роботу з великими обсягами структурованої інформації. Використання стандартних команд (SELECT, WHERE, GROUP BY) дозволяє ефективно витягувати та аналізувати дані безпосередньо з бази